

Tworzenie aplikacji z

TypeScript





Szkolenie

- Plan szkolenia i cele
- Aktualne obowiązki
- Pytania, dyskusje, potrzeby
- Elastyczny program szkoleniowy



Mateusz Kulesza

- Senior Software Developer
- Team Leader
- Project Manager
- Consultant and Trainer

Required software

Installation

<https://nodejs.org/en>

```
1 node -v
2 npm -v
```

GIT

<https://git-scm.com/download/win>

git -v

Alternatively NVM

<https://github.com/coreybutler/nvm-windows>

```
1 nvm install lts
2 nvm use 16.1.2
```

Editor choice

- Visual Studio Code
- IntelliJ WebStorm

Extensions

Visual Studio Code (Ctrl+Shift+X)

Prettier

<https://marketplace.visualstudio.com/items?itemName=esbenp.prettier-vscode>

Paste JSON as Code - quicktype

<https://marketplace.visualstudio.com/items?itemName=quicktype.quicktype>

Optional: Prettier TypeScript Errors, Tailwind, ESLint, ...

TypeScript

TypeScript to język programowania z składnią opartą na JS.

Dodaje statyczne i silne typowanie, aby umożliwić nam pisanie bezpieczniejszego i bardziej niezawodnego kodu.

TS kompiluje się głównie do JavaScriptu, ale może być również używany z WebAssembly (AssemblyScript) lub w środowiskach, gdzie proces kompilacji jest dla nas niezauważalny (ts-node, deno).

Po co nam TypeScript?

- Typowanie statyczne - mniej błędów
- Wnioskowanie typów - mniej pisania
- Dokumentacja i kontrakty - mniej zgadywania
- Lepsze narzędzia (Intellisense / podpowiedzi IDE) - produktywność
- Bezpieczna reużywalność

Javascript vs Typescript

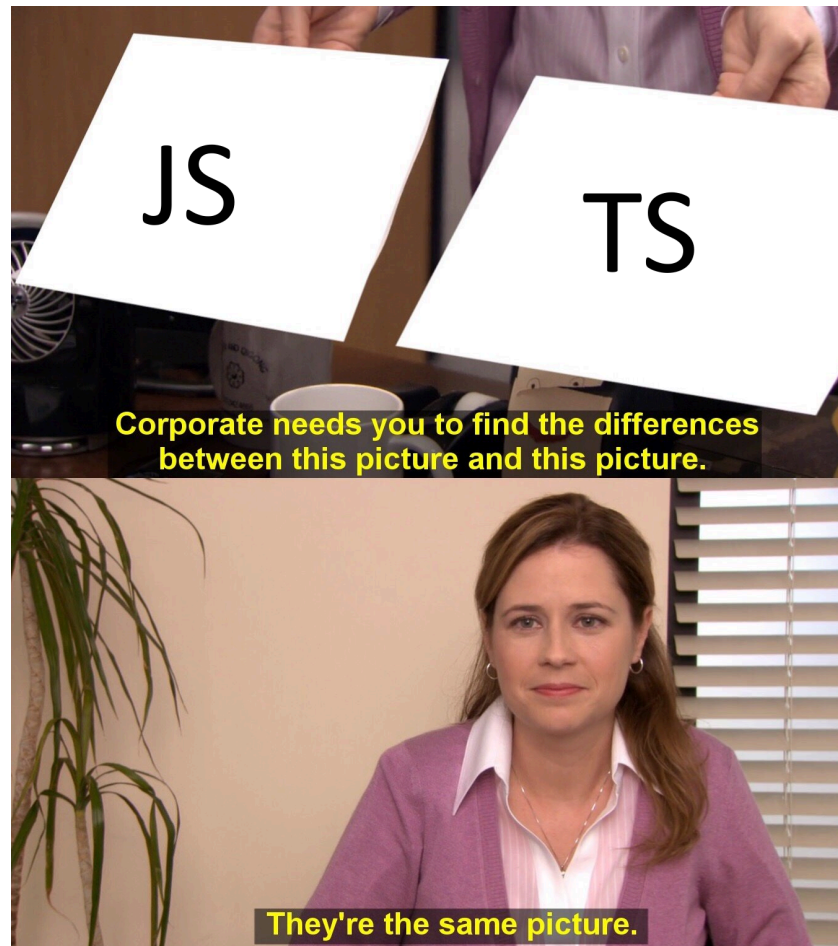
To jest kod JavaScript:

```
1  function add(a, b) {  
2    return a + b;  
3  }
```

To jest kod TypeScript*:

```
1  function add(a, b) {  
2    return a + b;  
3  }
```

* - bez włączonego trybu strict



JSDoc

Typowanie kodu JavaScript

```
1  /**
2   * Wyszukuje użytkownika po ID
3   *
4   * @param {string} id - Unikalny identyfikator użytkownika
5   * @returns {User | undefined} Obiekt użytkownika lub undefined
6   * @throws {NotFoundError} Gdy użytkownik nie istnieje
7   * @see {@link User} - Definicja typu User
8   *
9   * @example
10   * ```typescript
11   * const user = getUserById("123");
12   * if (user) {
13   *   console.log(user.name);
14   * }
15   * ```
16   */
17  function getUserById(id) {
18    return users.find(user => user.id === id);
19  }
```

Instalacja

tsc

Zainstaluj kompilator globalnie
(dla bieżącego użytkownika)

```
1 npm install -g typescript
```

powinien być dostępny przez zmienną środowiskową
\$PATH, więc wszędzie:

```
1 tsc --help
```

TypeScript

Kompilacja do pliku

```
1 tsc file.ts  
2  
3 tsc file.ts --watch
```

Dodawanie typów

```
1 function add(a: number, b: number) {  
2     return a + b;  
3 }
```

```
1 add(1, 2); // Ok  
2 add(1, "2"); // Błąd!  
3 add("1", "2"); // Błąd!  
4 add(); // Błąd!  
5 add([], {}); // Błąd!
```

Tryb STRICT

```
1 let value: number;  
2 value = null // ???  
3  
4 let anything; // ???
```

```
1 tsc --strict file.ts
```

TRYB STRICT

Tryb strict włącza następujące opcje kompilatora:

- `noImplicitAny` - Wyrażenia, które nie mają typu, nie staną się `any`, tylko spowodują błąd
- `noImplicitThis` - `this` bez określonego typu jest nieprawidłowe, zamiast stawać się `any`
- `alwaysStrict` - wszystkie pliki są rozumiane w trybie ECMAScript "use strict"
- `strictBindCallApply` - typy argumentów przekazywanych do `bind`, `call` i `apply` są dokładniej sprawdzane
- `strictNullChecks` - przestań akceptować `null` i `undefined` w zmiennych i argumentach jako prawidłowe wartości (jak w poprzednim przykładzie)
- `strictFunctionTypes` - argumenty funkcji nie są biwariantne
- `strictPropertyInitialization` - wszystkie pola klasy muszą być zainicjalizowane

Let vs Const

Wnioskowanie

```
1 let variable = 123;    // typ: number
2 variable = 999;        // OK
3
4 const constant = 123;   // typ: 123 (literal)
5 // constant = 456;     // Błąd!
```

Użycie `as const`

```
1 let fixedValue = 123 as const; // typ: 123
2 // fixedValue = 456;          // Błąd!
3
4 const config = {
5   type: 'SAVE' as const,
6   timeout: 5000
7 };
8 // config.type to "SAVE", nie string
```

Kluczowe Wnioski

- `let` wnioskuje szersze typy (number, string)
- `const` wnioskuje typy literalne (123, "hello")
- Użyj `as const` dla niemutowalnego wnioskowania z `let`

Typy Obiektów

Definiowanie struktury obiektów z określonymi właściwościami.

```
1  interface User {
2      id: string;
3      name: string;
4      color: "red" | "green" | "blue" | "orange";
5      pet?: {
6          // właściwość opcjonalna
7          name: string;
8      };
9  }
10
11  const users: User[] = [
12      { id: "123", name: "Alice", color: "pink" }, // Błąd! "pink" nie jest dozwolone
13      { id: "456", name: "Bob", color: "blue" }, // Ok
14  ];
```

Właściwości Opcjonalne

Oznaczanie właściwości, które mogą nie istnieć.

```
1  interface User {
2      id: string;
3      name: string;
4      pet?: {
5          name: string;
6          type: string;
7      };
8  }
9
10 // Optional chaining - bezpieczny dostęp
11 const petName = user.pet?.name; // string | undefined
12
13 // Nullish coalescing - wartość domyślna
14 const name = user.pet?.name ?? "Brak pupila";
```


Typy Unii

(Union Types)

Typ unii reprezentuje "zbiór możliwości" - wartość może być jednym z kilku typów.

```
1  function normalize(id: string | number): string {
2    if (typeof id === "string") {
3      return id;
4    }
5    return id.toString();
6  }
7
8  normalize("abc-123"); // Ok
9  normalize(42); // Ok
10 normalize(true); // Błąd!
```

Typy Literalne

(Literal Types)

Ograniczenie wartości do konkretnych stałych.

```
1  type Color = "red" | "green" | "blue" | "orange";
2  type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";
3  type Port = 3000 | 8080 | 443;
4
5  let color: Color = "red"; // Ok
6  color = "pink"; // Błąd!
7
8  let method: HttpMethod = "GET"; // Ok
9  method = "PATCH"; // Błąd!
```

Zawężanie Typów

(Type Narrowing)

TypeScript automatycznie zawęża typy na podstawie sprawdzeń.

```
1  function process(value: string | number) {  
2    if (typeof value === "string") {  
3      // Tutaj TypeScript wie, że value to string  
4      return value.toUpperCase();  
5    }  
6    // Tutaj TypeScript wie, że value to number  
7    return value.toFixed(2);  
8  }
```

Techniki zawężania:

- `typeof` - dla typów prymitywnych
- `instanceof` - dla instancji klas
- `in` - dla właściwości obiektów
- `===`, `!==` - dla wartości literalnych

Strażnicy Typów

(Type Guards) - `in` Operator

Sprawdzanie obecności właściwości w obiekcie.

```
1  type Options = { name: string } | { color: string } | { pet: string };
2
3  function queryUsers(options: Options): User[] {
4    if ("name" in options) {
5      // options.name jest dostępne
6      return users.filter((u) => u.name === options.name);
7    }
8    if ("color" in options) {
9      // options.color jest dostępne
10     return users.filter((u) => u.color === options.color);
11   }
12   // options.pet jest dostępne
13   return users.filter((u) => u.pet?.name === options.pet);
14 }
```

Przeciążenia Funkcji (Function Overloads)

Definiowanie różnych sygnatur dla tej samej funkcji.

```
1  /** Wyszukiwanie po nazwie */
2  function queryUsers(options: { name: string }): User[];
3  /** Wyszukiwanie po kolorze */
4  function queryUsers(options: { color: string }): User[];
5  /** Wyszukiwanie po pupiułu */
6  function queryUsers(options: { pet: string }): User[];
7
8  // Implementacja
9  function queryUsers(
10     options: { name: string } | { color: string } | { pet: string }
11 ): User[] {
12     // ... implementacja
13 }
```

Strażnicy Typów

Tworzenie funkcji sprawdzających typy.

```
1  type UserColor = "orange" | "pink" | "red" | "blue"
2
3  function isValidColor(color: string): color is UserColor {
4      return validColors.includes(color as any);
5  }
6
7  const input = "pink";
8  if (isValidColor(input)) {
9      // TypeScript wie, że input to UserColor
10     const color: UserColor = input;
11 }
```

Asercje `const`

Tworzenie typów tylko do odczytu z literalnych wartości.

```
1 // Bez as const
2 const colors = ["orange", "pink"];
3 // type: string[]
4
5 // Z as const
6 const colors = ["orange", "pink"] as const;
7 // type: readonly ["orange", "pink"]
8
9 type UserColor = (typeof colors)[number];
10 // type: "orange" | "pink"
```

Typ never

Typ reprezentujący wartości, które nigdy nie wystąpią.

```
1  function throwError(message: string): never {
2      throw new Error(message);
3  }
4
5  function infiniteLoop(): never {
6      while (true) {
7          // ...
8      }
9  }
```

Sprawdzanie Wyczerpania

(Exhaustiveness Checking)

```
1  // Użycie z zawężaniem
2  function process(value: string | number) {
3
4      if (typeof value === "string") {
5          return value.toUpperCase();
6      } else if (typeof value === "number") {
7          return value.toFixed(2);
8      }
9
10     // value ma typ never
11     // - wszystkie przypadki obsłużone
12     const _exhaustive: never = value;
13 }
```


Unie Dyskryminowane

Upewnienie się, że wszystkie przypadki zostały obsłużone.

```
1  type Media = Track | Episode | Playlist;
2
3  function getNumber(media: Media): number {
4      switch (media.type) {
5          case "track":
6              return media.track_no;
7          case "episode":
8              return media.episode_no;
9          case "playlist":
10             return media.tracks?.length ?? 0;
11         default:
12             // Jeśli zapomnimy o przypadku, TypeScript zgłosi błąd
13             const _exhaustive: never = media;
14             throw new Error("Nieobsłużony typ media");
15     }
16 }
```

Unie vs Przecięcia

```
1 // UNIA (OR) - wartość może być A LUB B
2 type Union = A | B;
3
4 const value1: Union = { ...tylko właściwości A }; // Ok
5 const value2: Union = { ...tylko właściwości B }; // Ok
6 const value3: Union = { ...właściwości A i B }; // Ok
```

```
1 // PRZECIĘCIE (AND) - wartość musi być A I B
2 type Intersection = A & B;
3
4 const value4: Intersection = { ...tylko A }; // Błąd!
5 const value5: Intersection = { ...tylko B }; // Błąd!
6 const value6: Intersection = { ...A i B }; // Ok
```

Object, {} czy object ?

Jak typować "obiekt"?

Typowanie `object`

Dowolna instancja obiektu

```
1  function showObject(params: object) {  
2      // Wyklucza prymitywy, null, undefined  
3  }  
4  showObject({});           // ✓  
5  showObject(new Date());   // ✓  
6  // showObject(123);       // ✗
```

Typowanie Object

Z metodami prototypu

```
1  function showObject(params: Object) {  
2      params.hasOwnProperty("x"); // Dostępne  
3  }  
4  showObject(123);                // ✓ (opakowane)  
5  showObject({});                 // ✓
```

Typowanie {}

Dowolny obiekt nie-null

```
1  function showObj(params: {}) {  
2    // Brak widocznego prototypu  
3  }  
4  showObj(123);           // ✓  
5  showObj({});           // ✓  
6  // showObj(null);      // X
```

Typowanie Strukturalne

vs Nominalne

Duck Typing

"Jeśli kwacze jak kaczką..."

```
1  type Point = { x: number; y: number };
2  type Vector = { x: number; y: number; length: number };
3
4  let p: Point = { x: 1, y: 2 };
5  let v: Vector = { x: 1, y: 2, length: 5 };
6
7  p = v; // ✓ Vector ma wszystkie właściwości Point
8  // v = p; // ✗ Point brakuje 'length'
```

Rozszerzanie Typu

```
1  if ("length" in p && typeof p.length === "number") {
2    const v: Vector = p;
3    v.length; // TypeScript wie, że p to jednak Vector
4  }
```


Kowariancja

Daj Co Najmniej To, Czego Oczekiwano

```
1  function fromPoint(p: Point) {}  
2  
3  fromPoint(p); // ✓  
4  fromPoint(v); // ✓ Vector też działa!
```

Kontrawariancja

Akceptuj To, Co Otrzymano lub Mniej

```
1  type Renderer = {  
2    renderVector: (v: Vector) => void;  
3    renderPoint: (p: Point) => void;  
4  };  
5  
6  renderer.renderVector = renderPoint; // ✓ Point akceptuje mniej  
7  // renderer.renderPoint = renderVector; // ✗ Nie może wymagać więcej
```

Tablice i Krotki

Typy Tablic

```
1  const array: number[] = [1, 2, 3, 4];  
2  const names: Array<string> = ["Alice", "Bob"];  
3  
4  const mixed = [123, "text", true]; // (string | number)
```

Typy Krotek

```
1  type Person = [id: number, name: string];  
2  
3  let person: Person = [123, "Pankracy"];  
4  
5  function takePerson( ...person: Person) {}  
6  takePerson(123, "Alice"); // ✓
```

Bezpieczna Typowo Siatka

```
1  type Field = "X" | "O" | "";
2
3  type TicTacToe = [
4    [Field, Field, Field],
5    [Field, Field, Field],
6    [Field, Field, Field]
7  ];
8
9  const game: TicTacToe = [
10   ["X", "O", "O"],
11   ["O", "X", "X"],
12   ["X", "X", "O"]
13  ];
```

Literaty

```
1  type Modes = "details" | "editor" | "creator";  
2  
3  let mode: Modes = "details"; // ✓  
4  // mode = "invalid";         // ✗
```

Szablonowe Typy Literałów

```
1  type Protocol = "http" | "https";
2  type Domain = "zus.pl" | "zus.ad";
3
4  type URL = `${Protocol}://${Domain}/`;
5
6  const url: URL = "https://zus.pl/"; // ✓
```


Zaawansowany Przykład

z Nagłówkami

```
1  type Header = "Authorization" | "Content-Type";
2  type ExtraHeaders = `X-${string}`;
3  type ExtendedHeaders = Header | ExtraHeaders;
4
5  function setHeader(header: ExtendedHeaders) {}
6
7  setHeader("X-Proxy");           // ✓
8  setHeader("Authorization");     // ✓
9  // setHeader("Cookies");        // X
```

Typeof i Keyof

typeof

Wyciąganie Typów Wartości

```
1  const UserRoles = {
2    USER: "user",
3    ADMIN: "admin",
4    MODERATOR: "moderator"
5  } as const;
6
7  type Roles = typeof UserRoles;
8  // { USER: "user", ADMIN: "admin", MODERATOR: "moderator" }
9
10
11 // Wyciąganie typu z tablicy
12 const users: User[] = [];
13 type User = (typeof users)[number]; // User
```

keyof

Wyciąganie Kluczy Obiektu

```
1  type RoleKeys = keyof Roles;  
2  // "USER" | "ADMIN" | "MODERATOR"
```

Łączenie `typeof`, `keyof` i Wyszukiwania

```
1  type RoleValues = (typeof UserRoles)[keyof typeof UserRoles];  
2  // "user" | "admin" | "moderator"  
3  
4  const role: RoleValues = "admin"; // ✓
```

Typy Konstruktor vs Instancji

```
1  class Pancake {  
2      flavor = "sweet";  
3  }  
4  
5  const pancake = new Pancake();  
6  
7  type Instance = typeof pancake;    // Pancake  
8  type Constructor = typeof Pancake; // typeof Pancake
```

Typy Generyczne <T>

Typy-Pudełka

Podstawowe Funkcje Generyczne

```
1  function identity<T>(x: T): T {  
2      return x;  
3  }  
4  
5  const num = identity(123);    // T = number  
6  const str = identity("hello"); // T = string
```


Ograniczenia Generyczne

```
1  function getUserInfo<T extends { name: string }>(person: T): T["name"] {  
2      return person.name;  
3  }  
4  
5  getUserInfo({ name: "Alice" });           // ✓  
6  getUserInfo({ name: "Bob", age: 30 });    // ✓  
7  // getUserInfo({ age: 30 });              // ✗
```

Dlaczego Używać Generyków?

```
1 // X Traci informację o typie
2 function logAndReturnPerson(person: { name: string }) {
3     console.log(person.name);
4     return person;
5 }
6
7 const bot = { name: "ChatGPT", model: "gpt-4" };
8
9 logAndReturnPerson(bot).model; // Błąd - model nie istnieje
```

```
1 // ✓ Zachowuje informację o typie
2 function logAndReturnPerson2<T extends { name: string }>(person: T): T {
3     console.log(person.name);
4     return person;
5 }
6
7 logAndReturnPerson2(bot).model; // ✓ "gpt-4"
```

Typy Indeksowane

oraz Mapowane i Inferowane

Dynamiczny Dostęp do Właściwości

```
1  type CacheItem = {  
2      name: string;  
3  };  
4  
5  // Typ Indeksowany  
6  type Cache = { [key: string]: CacheItem };  
7  
8  const cache: Cache = {  
9      "123": { name: "playlist 123" },  
10     "345": { name: "playlist 345" }  
11 };  
12  
13 cache["999"] = { name: "new item" }; // ✓
```

Bezpieczny

vs Niebezpieczny Dostęp

```
1 // Niebezpieczny - może zwrócić undefined
2 type Cache = { [key: string]: CacheItem };
3
4 cache["does not exist"].name; // Błąd w runtime!
5
6 // Bezpieczny - jawny undefined
7 type SafeCache = {
8   [key: string]: CacheItem | undefined
9 };
10
11 const item = cache["123"];
12 if (item) {
13   item.name; // ✓ Bezpieczne
14 }
```

Alternatywna Składnia

```
1 type SafeCache2 = {
2   [key in string]?: CacheItem
3 };
```

Typy Mapowane

```
1  type HeadersList =  
2    | "Accept"  
3    | "Authorization"  
4    | `X-${string}`;  
5  
6  type RequestConfig = {  
7    headers: {  
8      [headerName in HeadersList]: string;  
9    };  
10 };  
11  
12 const req: RequestConfig = {  
13   headers: {  
14     Accept: "application/json",  
15     Authorization: "Bearer token",  
16     "X-Proxy": "custom"  
17   }  
18 };
```

Mapowanie z Wyszukiwaniem

```
1  type events = "click" | "change";
2  type reactEvents = `on${Capitalize<events>}`; // "onClick" | "onChange"
3
4  type EventTypes = {
5    onClick: React.MouseEvent;
6    onChange: React.ChangeEvent;
7  };
8
9  type ComponentEventProps = {
10    [e in reactEvents]: React.EventHandler<EventTypes[e]>;
11  };
```

Wbudowane Narzędzia

```
1  type Partial<T> = { [K in keyof T]?: T[K] };
2  type Readonly<T> = { [K in keyof T]: readonly T[K] };
3  type Pick<T, K extends keyof T> = { [P in K]: T[P] };
```


Typy Warunkowe

```
1  type Point = { x: number; y: number };
2  type Vector = { x: number; y: number; length: number };
3
4  type IsVectorAPoint = Vector extends Point ? true : false; // true
5  type IsPointAVector = Point extends Vector ? true : false; // false
```

Dystrybucja nad Uniami

```
1  type ExcludeBanana<T> = T extends "banana" | "bananas" ? never : T;  
2  
3  type NoBananas = ExcludeBanana<"bananas" | "banana" | "apple" | "placki">;  
4  // "apple" | "placki"
```

Wbudowane Narzędzia Warunkowe

```
1  type Exclude<T, U> = T extends U ? never : T;  
2  type Extract<T, U> = T extends U ? T : never;  
3  
4  type OnlyStrings = Extract<"banana" | "placki" | 123 | true, string>;  
5  // "banana" | "placki"
```

Własny Omit

z Typami Warunkowymi

```
1  type Omit<T, K> = Pick<T, Exclude<keyof T, K>>;  
2  
3  type Point2D = Omit<Vector, "length">;  
4  // { x: number; y: number }
```

Typy Wnioskowane

Słowo Kluczowe `infer`

```
1  type Parameter<F> = F extends (p: infer P) ⇒ any ? P : never;
2  type Parameters<F> = F extends ( ...params: infer P) ⇒ any ? P : never;
3
4  function fetchData(id: number, options: { cache: boolean }) {
5    // ...
6  }
7
8  type FirstParam = Parameter<typeof fetchData>; // number
9  type AllParams = Parameters<typeof fetchData>; // [number, { cache: boolean }]
```

Wyciąganie Typów Zwracanych

```
1  type ReturnType<F> = F extends ( ...args: any[] ) => infer R ? R : never;
2
3  function fetchPlaylists() {
4    return Promise.resolve([ { id: 1, name: "Rock" } ]);
5  }
6
7  type FetchReturn = ReturnType<typeof fetchPlaylists>;
8  // Promise<{ id: number; name: string }[]>
```

Rozpakowywanie Promises

```
1  type Awaited<T> = T extends Promise<infer U> ? U : T;  
2  
3  type Data = Awaited<FetchReturn>;  
4  // { id: number; name: string }[]
```


Zagnieżdżona Ekstrakcja

```
1  type GeolocationCallback = Parameters<
2    typeof navigator.geolocation.getCurrentPosition
3  >[0];
4
5  type Position = Parameters<GeolocationCallback>[0];
6  type Coords = Position["coords"];
```